

SISTEMAS OPERATIVOS

3004610 - 1

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Sincronización de hilos

Contenido

1. El problema: hilos que comparten datos y condiciones de carrera
2. La sección crítica: condiciones y soluciones por software
3. Semáforos: operaciones P y V
4. Mutex en pthreads: uso y ejemplos



El problema: datos compartidos

Hilos que comparten una variable



- El hilo principal crea hilos trabajadores con `pthread_create`; estos hacen su trabajo y terminan con `pthread_exit`; el principal los recoge con `pthread_join`.
- Todos los hilos del proceso **comparten las variables globales y el heap**: es su forma natural de comunicarse.
- Si dos hilos leen y escriben la **misma variable** sin coordinarse, el resultado depende del orden en que se ejecuten: **condición de carrera**.
- **Sincronizar** es coordinar a los hilos para que el acceso a los datos compartidos sea correcto.

Ejemplo 1: dos hilos incrementan sum

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void *funcion_hilo(void *param);
void algo(int);
int sum;

int main(void) {
    int i;
    pthread_t thread_id;

    sum = 0;
    printf("Previa creacion de hilos sum=%d\n", sum);

    pthread_create(&thread_id, NULL, funcion_hilo, NULL);
    for (i = 0; i < 10000; i++) {
        algo(16);
        sum++;
    }
    pthread_join(thread_id, NULL);
    printf("Hilo principal sum=%d\n", sum);
    return 0;
}
```

```c
void *funcion_hilo(void *param) {
    int i;
    (void) param;
    printf("Hilo %lu\n", (unsigned long) pthread_self());

    for (i = 0; i < 10000; i++) {
        sum++;
        algo(16);
    }
    pthread_exit(NULL);
}

void algo(int n) { usleep(n); }
```

```
$ gcc -Wall -pthread -o carrera carrera.c
$ ./carrera
Previa creacion de hilos sum=0
Hilo 134875629745856
Hilo principal sum=19772
$ ./carrera
Previa creacion de hilos sum=0
Hilo 138165358687936
Hilo principal sum=19579
$ ./carrera
Previa creacion de hilos sum=0
Hilo 134734277506752
Hilo principal sum=19397
```

Cada hilo suma 10000 veces: esperamos `sum = 20000`, pero el resultado cambia en cada ejecución y casi nunca llega a 20000. ¿Por qué?

sum++ no es una operación atómica

sum++	=	sum = sum + 1	=

```
mov ax, [sum]   ; 1. leer
add ax, 1       ; 2. sumar 1
mov [sum], ax   ; 3. escribir
```

Tres instrucciones de máquina: el hilo puede ser interrumpido entre ellas. Cada hilo tiene su propia copia de los registros (ax).

Paso	Hilo 1	Hilo 2	sum en memoria
1	`mov ax, [sum]` → ax = 5		5
2		`mov ax, [sum]` → ax = 5	5
3	`add ax, 1` → ax = 6		5
4		`add ax, 1` → ax = 6	5
5	`mov [sum], ax`		6
6		`mov [sum], ax`	**6**

Dos incrementos, pero `sum` solo pasó de 5 a 6: **se perdió una actualización**. Con 10000 iteraciones por hilo, cada entrelazado desafortunado pierde un incremento; por eso el resultado es menor que 20000 y distinto en cada ejecución.



La sección crítica y sus soluciones

El problema de la sección crítica

- **Sección crítica**: segmento de código en el que el hilo (o proceso) accede a datos compartidos.
- Toda solución debe cumplir tres condiciones:
  - **Exclusión mutua**: si el proceso P_i está ejecutando su sección crítica, ningún otro proceso puede ejecutar la suya.
  - **Progreso**: si ningún proceso está en su sección crítica y hay procesos que desean entrar, la decisión no puede posponerse indefinidamente.
  - **Espera limitada**: hay un límite al número de veces que otros procesos pueden entrar en su sección crítica antes de que a un proceso que espera se le conceda la entrada.

Esquema general de solución:

```
do {
    protocolo de entrada
    sección crítica
    protocolo de salida
    resto de la sección
} while (TRUE);
```

El protocolo de entrada pide permiso para entrar en la sección crítica; el de salida avisa que ya se salió. Cada proceso repite este ciclo.

Solución 1: alternancia estricta (turno)

Variables compartidas:

```
int turno;      /* vale i o j: quién puede entrar */
```

```
do {
    while (turno != i) nada();
    sección crítica
    turno = j;
    resto de la sección
} while (TRUE);
```

En C real: una variable consultada en espera activa debe ser al menos `volatile` (si no, el compilador puede leerla una sola vez y el `while` no termina). `volatile` no la vuelve atómica ni sincroniza: para eso están las operaciones atómicas, semáforos y mutex.

- **Idea:** los dos procesos P_i y P_j se turnan: P_i solo entra cuando `turno == i` y al salir cede el turno al otro.
- **Cumple exclusión mutua:** nunca están los dos en la sección crítica.
- **Defecto: no cumple progreso.** Si P_j no quiere entrar, P_i no puede entrar dos veces seguidas: queda esperando un turno que nadie le cede.
- **Además:** espera activa (*busy waiting*): el proceso consume CPU mientras espera en el `while`.

Solución 2: banderas de intención

Variables compartidas:

```
boolean listo[2];      /* listo[i] = listo[j] = FALSE */
```

```
do {
    listo[i] = TRUE;
    while (listo[j]) nada();
    sección crítica
    listo[i] = FALSE;
    resto de la sección
} while (TRUE);
```

- **Idea:** cada proceso anuncia que quiere entrar (`listo[i] = TRUE`) y espera mientras el otro también quiera.
- **Cumple exclusión mutua:** un proceso entra solo si el otro no está listo.
- **Defecto: no cumple progreso.** Si ambos ponen su bandera en TRUE casi a la vez, cada uno espera al otro para siempre: **interbloqueo**.
- Si se invierte el orden (esperar primero y anunciar después) se pierde la exclusión mutua.

Solución 3: algoritmo de Peterson

Variables compartidas:

```
int turno;              /* 0 o 1 */
boolean listo[2];       /* listo[i] = listo[j] = FALSE */
```

```
do {
    listo[i] = TRUE;
    turno = j;
    while (listo[j] && turno == j) nada();
    sección crítica
    listo[i] = FALSE;
    resto de la sección
} while (TRUE);
```

- **Idea:** combina las dos anteriores: anuncia su intención y cede el turno; espera solo si el otro quiere entrar y es su turno.
- **Cumple las tres condiciones:** exclusión mutua, progreso y espera limitada (Peterson, 1981).
- **Limitaciones:** espera activa; solo dos procesos; supone que las lecturas y escrituras en memoria son atómicas y en orden, algo que los procesadores modernos no garantizan.
- En la práctica se usan mecanismos del hardware y del SO: instrucciones atómicas, **semáforos** y **mutex**.



Semáforos

Semáforos: operaciones P y V

- Un **semáforo** es un tipo especial de variable entera con dos únicas operaciones, **P** y **V**, ambas **atómicas** (propuesto por Dijkstra).
- **P(s)** (*proberen*, probar; también *wait*): espera hasta que el semáforo sea positivo y, en ese momento, lo decrementa en 1.
- **V(s)** (*verhogen*, incrementar; también *signal*): incrementa el semáforo en 1.
- Para proteger una sección crítica se usa un semáforo inicializado en 1: **P** antes de entrar y **V** al salir.

```
P(sem);   // probar
    sección crítica
V(sem);   // incrementar
```

Estructura de un semáforo:

```
typedef struct {
    int contador;
    cola q;    /* procesos en espera */
} SEMAFORO;
```

Las operaciones son **atómicas**: el sistema operativo garantiza que P y V no se interrumpen a la mitad; así el propio contador del semáforo no sufre condiciones de carrera.

Implementación de P y V

```
P(SEMAFORO *s) {
    s->contador -= 1;
    if (s->contador < 0) {
        añadir el proceso a s->q;
        bloquear();
    }
}
```

```
V(SEMAFORO *s) {
    s->contador += 1;
    if (s->contador <= 0) {
        sacar un proceso k de s->q;
        despertar(k);
    }
}
```

- Si `contador` es negativo, su valor absoluto es el número de procesos bloqueados en la cola `q`.
- `bloquear` y `despertar` las provee el sistema operativo: el proceso que espera no consume CPU (no hay espera activa).
- P y V deben ser atómicas: el SO lo garantiza deshabilitando interrupciones o con instrucciones atómicas del procesador.
- Un proceso despertado por V continúa justo después de su P: entra en la sección crítica que quedó libre.

Semáforos binarios y contadores

Tipo	Valor inicial	Uso	Ejemplo
Binario	`1`	Exclusión mutua: solo un proceso a la vez en la sección crítica.	Proteger `sum++` en el ejemplo 1
Contador	`N > 1`	Hasta N procesos a la vez: control de un recurso con N instancias.	Un buffer con N posiciones libres
Contador	`0`	Sincronización: un proceso espera (P) a que otro señale (V) que ocurrió un evento.	Esperar a que otro hilo produzca un dato

- Según el valor inicial de `contador`, el semáforo puede permitir que varios procesos entren a la vez en la sección crítica, si se necesita esa posibilidad.
- **Doble uso** de los semáforos: **exclusión mutua** (proteger datos compartidos) y **sincronización** (ordenar eventos entre procesos o hilos).
- En POSIX, P y V se llaman `sem_wait` y `sem_post` (`<semaphore.h>`, se inicializan con `sem_init`); System V ofrece `semget/semop`.
- En pthreads, la exclusión mutua entre hilos se implementa con **mutex** (siguiente sección).



Mutex en pthreads

Mutex: exclusión mutua en pthreads

- **Mutex** = abreviatura de *mutual exclusion* (exclusión mutua).
- Es el principal medio para sincronizar hilos y proteger los datos compartidos cuando varios hilos escriben concurrentemente.
- Una variable mutex actúa como una **cerradura** (*lock*): varios hilos pueden intentar bloquearla, pero solo uno lo consigue.
- Los demás hilos quedan **bloqueados** (sin consumir CPU) hasta que el hilo propietario desbloquea el mutex.
- Equivale a un semáforo binario con una restricción: solo el hilo propietario puede liberarlo.

Secuencia típica de uso de un mutex

1. Crear e inicializar una variable mutex.
2. Varios hilos intentan bloquear (`lock`) el mutex.
3. Solo uno tiene éxito: ese hilo posee el mutex.
4. El propietario realiza sus acciones (sección crítica).
5. El propietario desbloquea (`unlock`) el mutex.
6. Otro hilo adquiere el mutex y repite el proceso.
7. Al final, el mutex se destruye.

Funciones para mutex en pthreads

Operación	Función	Descripción
Creación	*pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;* *int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);*	Inicialización estática (al declararla) o dinámica; `attr = NULL`: atributos por defecto.
Bloqueo	*int pthread_mutex_lock(pthread_mutex_t *mutex);*	Adquiere el mutex; si está tomado, el hilo se bloquea hasta que quede libre.
Intento	*int pthread_mutex_trylock(pthread_mutex_t *mutex);*	Como `lock`, pero no bloquea: si está tomado retorna EBUSY de inmediato.
Desbloqueo	*int pthread_mutex_unlock(pthread_mutex_t *mutex);*	Libera el mutex; debe llamarlo el hilo propietario.
Destrucción	*int pthread_mutex_destroy(pthread_mutex_t *mutex);*	Libera los recursos del mutex, que debe estar desbloqueado.

Todas retornan 0 si tienen éxito o un código de error. Incluir `<pthread.h>` y compilar con `gcc -pthread`. La sección crítica va entre `pthread_mutex_lock` y `pthread_mutex_unlock`.

Ejemplo 2: sum++ protegida con un mutex

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void *funcion_hilo(void *param);
void algo(int);
int sum;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main(void) {
    int i;
    pthread_t thread_id;

    sum = 0;
    printf("Previa creacion de hilos sum=%d\n", sum);

    pthread_create(&thread_id, NULL, funcion_hilo, NULL);
    for (i = 0; i < 10000; i++) {
        algo(16);
        pthread_mutex_lock(&mutex);
        sum++;
        pthread_mutex_unlock(&mutex);
    }
    pthread_join(thread_id, NULL);
    printf("Hilo principal sum=%d\n", sum);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

```c
void *funcion_hilo(void *param) {
    int i;
    (void) param;
    printf("Hilo %lu\n", (unsigned long) pthread_self());

    for (i = 0; i < 10000; i++) {
        pthread_mutex_lock(&mutex);
        sum++;
        pthread_mutex_unlock(&mutex);
        algo(16);
    }
    pthread_exit(NULL);
}
```

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
Previa creacion de hilos sum=0
Hilo 124461705590464
Hilo principal sum=20000
$ ./mutex
Previa creacion de hilos sum=0
Hilo 123580681549504
Hilo principal sum=20000
```

Con el mutex, `sum++` se ejecuta en exclusión mutua: el resultado es siempre 20000 (sin mutex: 19772, 19579, 19397).

Ejemplo 3: 20 hilos y una variable global

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define CHILOS 20
int global = 0;
void *funcion_hilo(void *);
void hacer_algo(int);

int main(void) {
    int i;
    pthread_t hilos[CHILOS];
    for (i = 0; i < CHILOS; i++) {
        pthread_create(&hilos[i], NULL, funcion_hilo, NULL);
    }
    for (i = 0; i < CHILOS; i++) {
        pthread_join(hilos[i], NULL);
    }
    printf("\n");
    return 0;
}
```

```c
void *funcion_hilo(void *args) {
    (void) args;
    hacer_algo(global);
    printf("%d ", global);
    global = global + 1;
    pthread_exit(NULL);
}

void hacer_algo(int n) {
    (void) n;        /* solo simula trabajo */
    usleep(10000);   /* 10 ms */
}
```

```
$ ./global20
0 0 0 0 0 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
$ ./global20
0 0 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
```

Error común: creer que las sentencias de C son atómicas. Entre la lectura de `global` y su incremento pueden ejecutarse otros hilos, aunque no haya ningún `usleep`.

Cada hilo lee `global`, duerme 10 ms, imprime y luego incrementa. Como nadie protege la secuencia leer-imprimir-incrementar, varios hilos leen el mismo valor: en la primera ejecución cinco hilos imprimen 0 y los valores 1 a 4 nunca aparecen; en la segunda, el 0 se repite y falta el 1. Con un mutex alrededor de las tres sentencias, cada hilo vería un valor distinto.

Ejemplo 4: 40 hilos toman un turno

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define CHILOS 40

int global = 0;
void *funcion_hilo1(void *);
void hacer_algo(long int);

int main(void) {
    int i;
    pthread_t hilos[CHILOS];
    for (i = 0; i < CHILOS; i++) {
        pthread_create(&hilos[i], NULL, funcion_hilo1, NULL);
    }
    for (i = 0; i < CHILOS; i++) {
        pthread_join(hilos[i], NULL);
    }
    printf("%d\n", global);
    return 0;
}
```

```c
void *funcion_hilo1(void *args) {
    int myturno;
    (void) args;
    hacer_algo(100);
    myturno = global;
    printf("%lu %d\n", (unsigned long) pthread_self(),
           myturno);
    global = global + 1;
    pthread_exit(NULL);
}

void hacer_algo(long int n) { usleep(n); }
```

Cada hilo toma como turno el valor actual de `global` y luego lo incrementa. Como nada protege esa secuencia, varios hilos leen el mismo valor antes de que otro lo incremente: aparecen **turnos repetidos** (0, 2, 32) y **turnos saltados** (3, 4, 33, 34).

Salida (primeras y últimas líneas):

```
$ ./turno40
140706232399552 0
140706215614144 1
140706224006848 0
140706198828736 2
140706207221440 2
140706190436032 5
...
140705844340416 36
140705852733120 37
140705835947712 38
140705827555008 39
40
```

Ejemplo 4 con mutex: un turno distinto para cada hilo

turno40_mutex.c (cambios respecto a turno40.c):

```
int global = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
...
void *funcion_hilo1(void *args) {
    int myturno;
    (void) args;
    hacer_algo(100);
    pthread_mutex_lock(&mutex);
    myturno = global;
    printf("%lu %d\n", (unsigned long) pthread_self(),
           myturno);
    global = global + 1;
    pthread_mutex_unlock(&mutex);
    pthread_exit(NULL);
}
```

Con el mutex, leer, imprimir e incrementar `global` forman una unidad (sección crítica): cada hilo recibe un turno distinto, de 0 a 39. El orden en que los hilos toman el mutex sigue dependiendo del planificador.

Sin mutex (últimas líneas):

```
140705911482048 29
140705903089344 30
140705894696640 31
140705869518528 32
140705877911232 32
140705886303936 32
140705861125824 35
140705844340416 36
140705852733120 37
140705835947712 38
140705827555008 39
40
```

Con mutex (últimas líneas):

```
135329897957056 29
135329889564352 30
135329881171648 31
135329872778944 32
135329864386240 33
135329855993536 34
135329847600832 35
135329839208128 36
135329830815424 37
135329822422720 38
135329814030016 39
40
```

- Sin mutex: el turno 32 se repite tres veces y los turnos 33 y 34 nunca aparecen.
- Con mutex: cada turno aparece una sola vez, en orden y sin huecos; la última línea (`40`) la imprime `main`.

Resumen

- Los hilos comparten las variables globales; si dos hilos modifican la misma variable sin coordinarse, el resultado depende del orden de ejecución: **condición de carrera**.
- `sum++` son tres instrucciones (leer, sumar, escribir): un cambio de hilo en medio pierde actualizaciones.
- Sección crítica: código que accede a datos compartidos. Toda solución debe garantizar exclusión mutua, progreso y espera limitada.
- Las soluciones por software (turno, banderas, Peterson) usan espera activa; solo el algoritmo de Peterson cumple las tres condiciones, y solo para dos procesos.
- Un semáforo es un entero con dos operaciones atómicas: P (esperar y decrementar) y V (incrementar). Con valor inicial 1 da exclusión mutua; con otros valores, sincroniza.
- En pthreads el mutex es la cerradura básica: `pthread_mutex_lock` antes de la sección crítica y `pthread_mutex_unlock` después; el resultado deja de depender del azar.

Referencias y lecturas

- Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*, 10.ª ed. Wiley, 2018. Capítulo 6: Synchronization Tools (sección crítica, algoritmo de Peterson, mutex, semáforos).
- Pacheco, P. S. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011. Capítulo 4: Shared-memory programming with Pthreads (condiciones de carrera, sección crítica, mutex).
- Peterson, G. L. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3), 1981.
- Dijkstra, E. W. Cooperating sequential processes, 1965 (semáforos y operaciones P y V).
- Barney, B. POSIX Threads Programming, Lawrence Livermore National Laboratory (modelo de hilo principal y trabajadores).
- Páginas de manual: `man 7 pthreads`, `man 3 pthread_mutex_init`, `man 3 pthread_mutex_lock`, `man 7 sem_overview`.